The `Main` function contains all the basic initialization code required to create a web server and to start listening for requests. It uses an `IHostBuilder`, created by the call to `CreateDefaultBuilder`, to define how the generic `IHost` is configured, before instantiating the `IHost` with a call to `Build()`.

TIP The `IHost` object represents your built application. The `WebApplication` type you've used throughout the book also implements `IHost`.

Much of your app's configuration takes place in the `IHostBuilder` created by the call to `CreateDefaultBuilder`, but it delegates some responsibility to a separate class, `Startup`. The `Startup` class referenced in the generic `UseStartup<>` method is where you configure your app's services and define your middleware pipeline.

NOTE The code to build the `IHostBuilder` is extracted to a helper method called `CreateHostBuilder`. The name of this method is historically important, as it was used implicitly by tooling such as the Entity Framework Core (EF Core) tools, as I discuss in section 30.5.

You may be wondering why you need two classes for configuration: `Program` and `Startup`. Why not include all your app's configuration in one class or the other? The idea is to separate code that changes often from code that rarely changes.

The `Program` class for two different ASP.NET Core applications typically look similar, but the `Startup` classes often differ significantly (though they all follow the same basic pattern, as you'll see in section 30.3). You'll rarely find that you need to modify `Program` as your application grows,